
Routes and Navigation

— Imperative Navigation —

Topics

Imperative Navigation

Navigating to/from a screen

Passing data to other screen

Returning data to the previous screen

Named Routes

onGenerateRoute

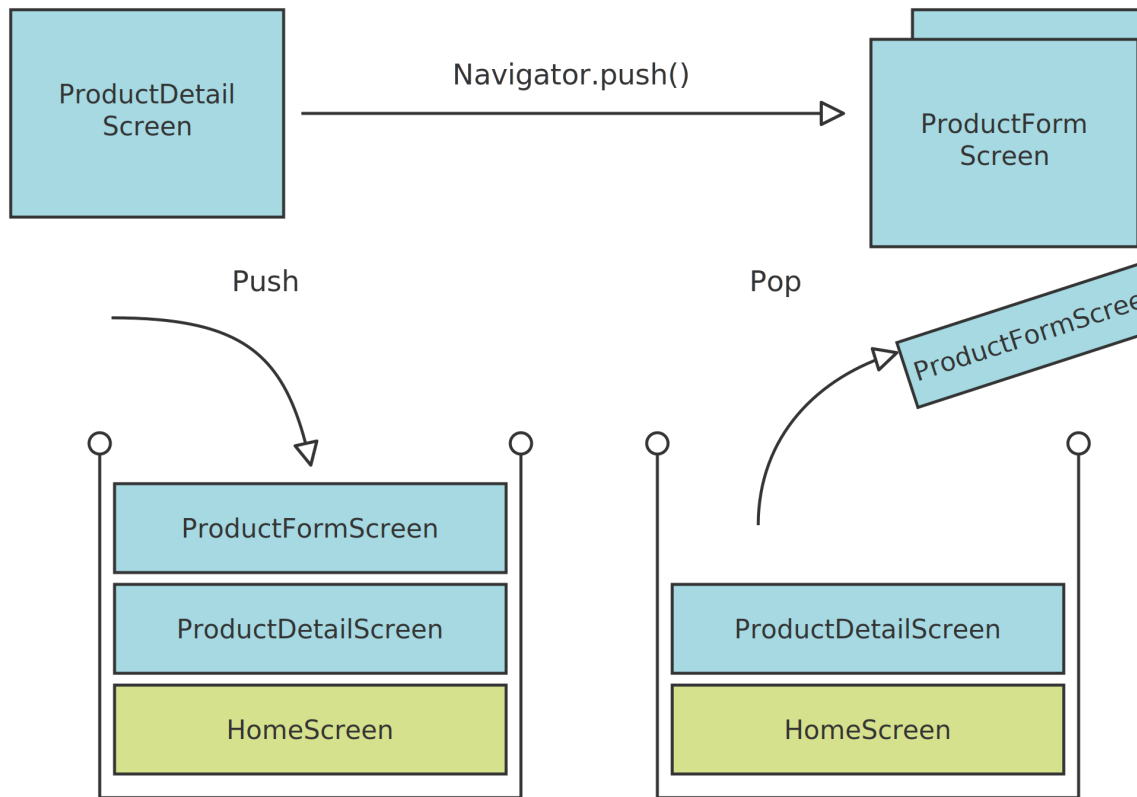
Introducing Navigator

In Flutter, you use a **Navigator** widget to manage your screens or pages

In Flutter, **screens** and **pages** are called **routes**

Introducing Navigator

Navigator uses stack-based scheme



Imperative Navigation overview

The most common imperative navigation APIs used to navigate between screens

`push()` : Adds a new route on the stack

`pop()` : Removes a route from the stack

<https://api.flutter.dev/flutter/widgets/Navigator-class.html>

Using Imperative Navigation

Most Flutter apps start with `WidgetsApp` as the root widget

`MaterialApp` extends `WidgetsApp`

`WidgetsApp` wraps, among other common widgets, a top-level `Navigator` widget to manage the pages you `push` and `pop`

Example

Suppose you have two screens - **HomeScreen**, **ProductDetailScreen** and the root app as shown in the next three slides

```
void main() {  
    runApp(const MiniMarketApp());  
}  
  
class MiniMarketApp extends StatelessWidget {  
    const MiniMarketApp({super.key});  
  
    @override  
    Widget build(BuildContext context) {  
        return MaterialApp(  
            title: 'Mini Market',  
            home: const HomeScreen(),  
        );  
    }  
}
```

```
class HomeScreen extends StatefulWidget {  
  const HomeScreen({super.key});  
  
  @override  
  State<HomeScreen> createState() => _HomeScreenState();  
}  
  
class _HomeScreenState extends State<HomeScreen> {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: const Text('Mini Market')),  
      body: const ProductGrid(),  
    );  
  }  
}
```

```
class ProductDetailScreen extends StatefulWidget {  
  final String productId;  
  const ProductDetailScreen({super.key, required this.productId});  
  
  @override  
  State<ProductDetailScreen> createState() => _ProductDetailState();  
}  
  
class _ProductDetailState extends State<ProductDetailScreen> {  
  @override  
  Widget build(BuildContext context) {  
    final product = MarketStore.findProduct(widget.productId);  
    return Scaffold(  
      appBar: AppBar(title: Text(product!.title)),  
      body: Text(product.description),  
    );  
  }  
}
```

Pushing routes

You can navigate to **ProductDetailScreen** from **HomeScreen** like so

```
Navigator.push(context, MaterialPageRoute(  
  builder: (context) => ProductDetailScreen(productId: product.id),  
));
```

```
class _HomeScreenState extends State<HomeScreen> {  
  /// Opens the detail screen and waits until we come back.  
  Future<void> _openProduct(Product product) async {  
    await Navigator.push(  
      context,  
      MaterialPageRoute(  
        builder: (context) => ProductDetailScreen(  
          productId: product.id,  
        ),  
      ),  
    );  
  
    setState(() {});  
  }  
}
```

Popping routes

You can navigate back to `HomeScreen` from `ProductDetailScreen` like so

```
Navigator.pop(context);
```

```
void _save() {  
    if (!_formKey.currentState!.validate()) {  
        return;  
    }  
  
    MarketStore.addProduct(  
        Product(  
            id: MarketStore.newProductId(),  
            title: _titleController.text.trim(),  
            price: double.parse(_priceController.text.trim()),  
            category: _category,  
            description: _descriptionController.text.trim(),  
        ),  
    );  
  
    Navigator.pop(context);  
}
```

Passing data to another screen

You can pass data

- using the widget's constructor

- using using **RouteSettings**

Passing data using RouteSettings

You can use **MaterialPageRoute** as shown below to pass data to the next screen

```
Navigator.push(context, MaterialPageRoute(  
  builder: (context) => const ProductDetailScreen(),  
  settings: RouteSettings(arguments: product.id),  
));
```


Using passed data

You can extract the passed data as shown below

```
final text =  
    ModalRoute.of(context)!.settings.arguments as String;
```

The full example showing passing and using data passed from **HomeScreen** to **ProductDetailScreen** is shown in the following two slides

Returning data from a screen

The following partial code shows a confirm dialog returning true or false to the screen that opened it

```
TextButton(  
  onPressed: () => Navigator.pop(context, false),  
  child: const Text('Cancel'),  
) ,  
TextButton(  
  onPressed: () => Navigator.pop(context, true),  
  child: const Text('Delete'),  
) ,
```

Accepting the returned result

The following partial code shows how to get the returned data from the other screen

```
final shouldDelete = await showDialog<bool>(
  context: context,
  builder: (context) => const ConfirmDeleteDialog(),
);
if (shouldDelete != true) return;
```

Navigate with named routes

Named route allows you to reuse navigation codes

To work with named routes you need to

- define routes

- use the `Navigator.pushNamed()` function

Defining named routes

```
class MiniMarketApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      initialRoute: '/',  
      routes: {  
        '/': (context) => const HomeScreen(),  
        '/cart': (context) => const CartScreen(),  
        '/add': (context) => const ProductFormScreen(),  
        '/product': (context) => const ProductDetailScreen(),  
      },  
    );  
  }  
}
```

Using named routes

Once the routes are defined, every screen opens by name

```
// open the cart
Navigator.pushNamed(context, '/cart');

// open a product, passing its id along
Navigator.pushNamed(
  context,
  '/product',
  arguments: product.id,
);
```

Named routes with `onGenerateRoute`

A more flexible way to handle named routes

It allows you to handle relatively complex situations such as parsing routes
an example is shown in the next slide

```
onGenerateRoute: (settings) {  
  if (settings.name == '/') {  
    return MaterialPageRoute(builder: (_) => const HomeScreen());  
  }  
  // '/product/p3'  
  final uri = Uri.parse(settings.name!);  
  if (uri.pathSegments.first == 'product') {  
    return MaterialPageRoute(  
      builder: (_) => ProductDetailScreen(  
        productId: uri.pathSegments[1],  
      ),  
    );  
  }  
  return MaterialPageRoute(builder: (_) => const UnknownScreen());  
},
```

Named routes with `onGenerateRoute`

An example showing the use of `onGenerateRoute` is shown in the next slide

```
class MiniMarketApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      initialRoute: '/',  
      onGenerateRoute: _generateRoute,  
    );  
  }  
}
```

```
Route<dynamic> _generateRoute(RouteSettings settings) {  
  final uri = Uri.parse(settings.name ?? '/');  
  if (uri.path == '/') {  
    return MaterialPageRoute(builder: (_) => const HomeScreen());  
  }  
  return MaterialPageRoute(  
    builder: (_) => ProductDetailScreen(productId: uri.pathSegments[1]),  
  );  
}
```

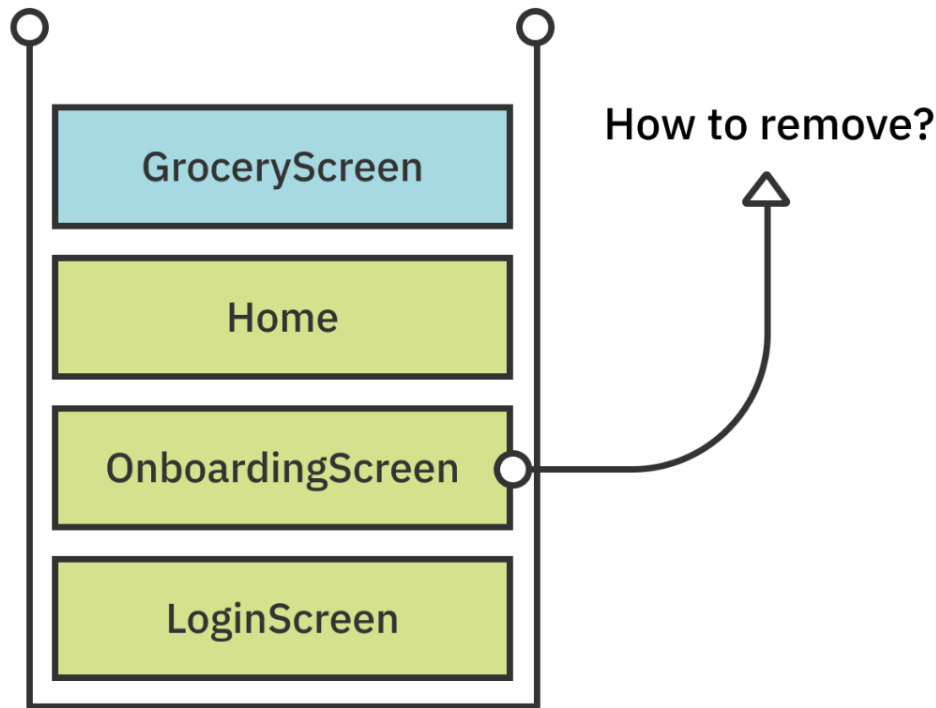
Limitations of Imperative Navigation

The API is imperative while Flutter itself is declarative

The imperative API does not allow developers to freely modify and rearrange the history stack to their liking

Limitations of Imperative Navigation

Difficult to adding and removing a screen between pages



Limitations of Imperative Navigation

Imperative navigation does not update the web URL path

Any time you go to a new page, you only see the base URL, like so:

www.localhost:8000/#/

Additionally, the web browser's forward and backward buttons may not work as expected

Limitations of Imperative Navigation

Imperative navigation does not update the web URL path

Any time you go to a new page, you only see the base URL, like so:

www.localhost:8000/#/

Additionally, the web browser's forward and backward buttons may not work as expected

Limitations of Imperative Navigation

On Android devices, the Back button might not work with imperative navigation when you have nested navigators

When To Use Imperative Navigation

For rapid prototyping or creating a small app for demos, the imperative API is suitable

Sometimes push and pop are all you need!

Mini Market with go_router

```
final router = GoRouter(  
  routes: [  
    GoRoute(  
      path: '/',  
      builder: (context, state) => const HomeScreen(),  
    ),  
    GoRoute(  
      path: '/product/:id',  
      builder: (context, state) => ProductDetailScreen(  
        productId: state.pathParameters['id']!,  
      ),  
    ),  
  ],  
);
```

Setting the location

go sets where the app is. push is only for screens that return a value.

```
// open a product from the grid
context.go('/product/${product.id}');

// a confirm screen that hands a result back
final ok = await context.push<bool>('/confirm');
```

Nesting means stacking

```
GoRoute(  
  path: '/',  
  builder: (context, state) => const HomeScreen(),  
  routes: [  
    GoRoute(  
      path: 'product/:id',  
      builder: (context, state) => ProductDetailScreen(  
        productId: state.pathParameters['id']!,  
      ),  
    ),  
  ],  
)
```

redirect: where state decides

```
final router = GoRouter(  
  refreshListenable: auth,  
  redirect: (context, state) {  
    final atSignIn = state.matchedLocation == '/sign-in';  
    if (!auth.signedIn && !atSignIn) return '/sign-in';  
    if (auth.signedIn && atSignIn) return '/';  
    return null;  
  },  
  routes: [ ... ],  
);
```

What to use when

push and pop: small apps, and any screen that hands a result back

go_router: deep links, web URLs, and sign-in rules

Rule of thumb - go for navigation, push only for results

Both end up on the same Navigator underneath

Quiz

What is the data structure that the imperative navigation API uses?

Write the basic code structure that you can use to Navigate to/from a screen